THOMPSON RIVERS UNIVERSITY

Department of Engineering

Faculty of Science

SENG 4620 - Practical Cloud Computing

Design Project: Building a Scalable and Global Cloud Infrastructure for a Local Business

This is a group project with a maximum of 3 students per team.

This semester, you will go on a journey to design, secure, and scale the cloud infrastructure for a mobile-first fintech startup called Aura Digital Bank. Transitioning the company from a fragile local prototype, you will guide Aura Digital Bank in deploying an enterprise-grade cloud architecture by translating business demands and strict regulatory standards (such as SOC2 and PCI-DSS) into robust technical solutions at each step. The project is divided into eight core engineering phases, each requiring verifiable evidence, architectural designs, and compliance artifacts as deliverables.

Learning Objectives

- To architect an enterprise-grade cloud infrastructure that meets evolving business requirements for scalability, cost-efficiency, and disaster recovery, while strictly adhering to regulatory compliance standards like SOC2 and PCI-DSS.
- To design and implement a zero-trust, multi-tier network topology by building a custom Virtual Private Cloud (VPC) that strictly isolates sensitive financial ledgers from public internet access.
- To define and deploy the entire core banking platform as Infrastructure-as-Code (IaC) using version-controlled modules and automated CI/CD pipelines to eliminate manual provisioning.
- To design a fault-tolerant architecture utilizing dynamic auto-scaling and load balancing to seamlessly handle massive, high-frequency user surges across multiple Availability Zones.
- To design and implement an isolated, event-driven serverless audit pipeline that securely analyzes database records and automatically alerts compliance teams to high-value transactions.
- To synthesize and defend complex cloud architectures, security topologies, and design decisions through an engineering handbook, verifiable audit evidence, and a recorded operational handover video.

Part 1: The Business Backstory & Stakeholder Mandates

Aura Digital Bank is a mobile-first fintech startup that offers secure, instant peer-to-peer (P2P) money transfers, digital checking accounts, and high-yield savings. To gain user trust and operate legitimately, the company must achieve regulatory clearance by aligning with strict SOC2 and PCI-DSS standards. Because the platform's original system is a fragile local prototype, you have been brought in to design and deploy an enterprise-grade cloud architecture. Your success in this project relies entirely on meeting the distinct technical mandates and business goals of four key stakeholders:

- **Elena (Chief Executive Officer):** Focused on minimizing infrastructure run costs, passing external audits, and expanding into international markets. She mandates a strict disaster recovery (DR) strategy to guarantee business continuity, even in the event of a complete primary cloud data center failure.
- **Marcus (Chief Technology Officer):** Requires a fully automated, scalable system with absolute repeatability. He insists on eliminating manual provisioning ("click-ops"), hardcoded configurations, and local data silos. Everything must be deployed programmatically, with a long-term goal of replacing monolithic application logic with modern microservices.
- **Siddharth (Lead Application Developer):** Prioritizes feature velocity and requires a secure, isolated sandbox for testing updates. He expects automated integration pipelines so his team can focus on writing code without having to manage servers or manually push updates.
- **Maya (Chief Information Security Officer):** Highly risk-averse and demands a strict Zero-Trust network topology. She mandates that the financial ledger database remain physically isolated from the public internet. Furthermore, all communication channels must be encrypted at rest and in transit, credentials must be systematically rotated, and all administrative and user actions must be fully audited.

Part 2: Core Engineering Phases

Phase 1: Brand Launch & Compliance Retention (Due: during lab 2)

Business Scenario Aura Digital Bank must publish its public-facing marketing portal, which includes compliance disclosures and interest rate schedules. At the same time, Elena (CEO) requires a highly secure storage mechanism to house sensitive, unmodifiable PDF customer onboarding forms.

Technical Requirements

To successfully complete this phase, you must:

- **Host the Marketing Portal:** Establish a secure, publicly accessible storage repository to host the static web assets.
- **Enforce WORM Storage:** Implement a write-once-read-many (WORM) storage pattern for the onboarding forms. These files must be strictly protected against accidental or malicious deletion, even by users with administrator privileges.
- **Optimize Costs with Lifecycle Policies:** Create automatic storage tiering policies that move stale compliance files and application logs to long-term, archival-grade storage after 90 days to control startup run-costs.

- **Implement Disaster Recovery:** Satisfy Elena's disaster recovery SLA by configuring automatic, real-time replication of all compliance documents to a geographically isolated region.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Live Endpoints:** The primary and replica public URLs of your storage endpoints.
- **Deletion Denial Log:** A formal verification log proving that a simulated attempt to delete a customer onboarding form was successfully blocked by system policy.
- **Access Policy Code:** The exact machine-readable access policy configured to allow public read-only access to the marketing assets while explicitly denying all other administrative operations to unauthorized networks.

Note: For the "public-facing marketing portal", you do not need to worry about web design, as UI/UX is completely out of scope for this course. You are free to design it however you wish, or simply create a basic index.html file with a single line of welcome text to prove your cloud storage can successfully serve a webpage. Similarly, we will not be providing actual legal documents for the "sensitive, unmodifiable PDF customer onboarding forms". You should simply create a dummy PDF file on your computer and upload it to test your storage policies.

To generate the "formal verification log" proving your WORM (Write-Once-Read-Many) setup works, log in with your administrator credentials and actively try to delete the dummy PDF you just uploaded. Take a clear screenshot of the resulting "Access Denied" or "Action Blocked" error message; this screenshot will serve as your official audit evidence. Finally, to satisfy the disaster recovery requirement, pick a primary deployment region (for example, US-East) and set up an automated replication rule targeting a completely different, "geographically isolated region" (such as US-West).

Phase 2: Core Ledger MVP & Machine Identity (Due: during lab 3)

Business Scenario Siddharth has delivered a dynamic monolithic PHP prototype of the core transactional ledger, and he requires a compute environment to execute this application. However, Maya (CISO) has flagged an immediate compliance violation: the prototype's database connection files currently contain hardcoded database passwords.

Technical Requirements

To successfully complete this phase, you must:

- **Deploy Compute Infrastructure:** Deploy a reliable compute environment to run the dynamic monolithic banking application.
- **Implement Secrets Management:** Establish an enterprise secrets management system to securely host database passwords and dynamic parameters.
- **Enforce Machine Identity:** Configure a secure, local machine identity that allows the compute server to retrieve secrets on-demand at application launch. Note: You must not use long-lived static IAM credentials or environment variables to achieve this.

- **Create a Golden Image:** Develop a "Golden Image" strategy that packages the baseline operating system, system dependencies, and application runtimes into a reusable baseline template.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Secrets Retrieval Log:** A screenshot or log demonstrating that the application successfully queries the system secrets manager at runtime to authenticate database connections.
- **IAM Policy Code:** The exact machine identity policy (IAM role/policy configuration) proving that only the designated compute instance is authorized to read the specific database secret.
- **Image Blueprint:** The configuration profile or blueprint script used to bake your dynamic "Golden Image".

Note:

Regarding the "dynamic monolithic PHP prototype", you are not expected to write a functional banking application or possess advanced software development skills. Simply create a basic "dummy" PHP script that is capable of querying your cloud provider's API to request a secret. Furthermore, because the actual relational database will not be deployed until Phase 3, you do not need to establish a real database connection right now. Your objective is strictly to store a dummy database password in your enterprise secrets management system and prove that your PHP script can successfully retrieve that specific secret at runtime using its local machine identity. Finally, for the "Golden Image" blueprint, you are free to use any method you prefer. You can utilize an automation tool like HashiCorp Packer, AWS EC2 Image Builder, or simply provide the user-data startup script you used to configure your baseline operating system before capturing your machine image.

Phase 3: The Vault (Relational Database Migration) (Due: during lab 4)

Business Scenario Aura's customer ledger is growing rapidly. Running the transactional database on the local disk of the application server is a catastrophic security risk and a major bottleneck. The database must be decoupled into an isolated tier that guarantees high availability and systematic backups.

Technical Requirements

To successfully complete this phase, you must:

- **Deploy a Managed Database:** Deploy a dedicated, managed relational database service configured to host the financial ledger.
- **Isolate from Public Internet:** Ensure the database service is completely isolated from direct public internet access.
- **Configure High Availability:** Implement a replication topology that guarantees the database can survive a physical data center outage (Availability Zone failure) without data loss or administrative intervention.
- **Execute Data Migration:** Migrate the database schema and customer accounts smoothly from the dynamic server launched in Phase 2 into this new, isolated relational tier.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Private Endpoint:** The private endpoint string of your managed database instance, providing proof that no public DNS entry exists.
- **Migration Proof:** Proof of a successful migration run (e.g., providing matching database row-count queries between the old dynamic instance and the new database vault).
- **Replication Status:** Verification logs or dashboard screenshots showing that database multi-datacenter replication is active and healthy.

Note: Regarding the requirement to "migrate the database schema and customer accounts", you do not need real financial data. Since a fully functional local database was not explicitly built in Phase 2, you should simply install a basic local database engine (such as MySQL, PostgreSQL, or SQLite) on your Phase 2 compute instance and create a simple table populated with a few rows of dummy customer data. Your objective is to extract this dummy data from your local compute instance and successfully import it into your newly deployed "managed relational database service". Furthermore, because you must "ensure the database service is completely isolated from direct public internet access", you will not be able to connect to the new database directly from your personal computer. You must route your database connections and execute your "matching database row-count queries" by first logging into your Phase 2 compute server and connecting to the database's "private endpoint string" from inside the cloud environment. You are free to choose whichever managed database service aligns with your chosen cloud provider (such as Amazon RDS).

Here is the revised version of **Phase 4**, keeping the same structured, checklist-style format for your students:

Phase 4: Perimeter Defense & Multi-Tier Isolation (Due: Sunday of Week7)

Business Scenario A security audit has highlighted that your application compute server is still exposed directly to the public internet, and administrative control channels (such as SSH/RDP) have been left open. You must design a zero-trust virtual network topology to effectively shield Aura's assets.

Technical Requirements

To successfully complete this phase, you must:

- **Build a Custom VPC:** Design a custom, isolated virtual network (VPC) from scratch using a strict IP addressing schema.
- **Enforce 3-Tier Architecture:** Segment the network to enforce a logical 3-tier architecture consisting of: Public Subnets (for load balancing), Private Subnets (for application logic), and Isolated Private Subnets (for the database engine).
- **Enable Private Internet Access:** Provide a secure, private mechanism for application servers inside the private subnets to pull operating system patches and security updates from the internet without exposing them to incoming connection requests.
- **Create an Administrative Gateway:** Establish a secure administrative gateway (such as a bastion host or a secure sessions manager) that allows authorized engineers to manage resources without exposing administrative ports to the public web.

- **Configure Firewalls:** Implement stateful and stateless firewalls to enforce strict least-privilege lateral communication between the network tiers.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Network Diagram:** A comprehensive logical network diagram illustrating the subnet IP ranges (CIDRs), route tables, gateways, and the traffic flow directions between the three tiers.
- **Traffic Matrix:** A Security Group/NACL traffic matrix showing that the database tier blocks all traffic except incoming traffic originating explicitly from the application server tier on the designated database port.

Note: Regarding the requirement to "design a custom, isolated virtual network (VPC) from scratch using a strict IP addressing schema", you are expected to define your own non-overlapping CIDR blocks (for example, creating a 10.0.0.0/16 network with /24 subnets). Because you are building this environment from scratch in this phase, be aware that you will need to either carefully migrate your existing compute instance and managed database from previous phases into this new VPC, or redeploy them entirely within your new boundaries. For the "secure, private mechanism" that allows outbound internet access for patches, you are free to utilize a managed NAT Gateway or configure a NAT instance. When establishing the "secure administrative gateway", you may provision a traditional Bastion Host in your public subnet or opt for a modern, portless alternative like AWS Systems Manager (SSM) Session Manager. Finally, no specific software is required for the "comprehensive logical network diagram"; you may use any standard diagramming tool (such as draw.io, Lucidchart, or Visio) as long as it clearly maps out your custom IP assignments, gateways, and the specific stateful (Security Groups) and stateless (Network ACLs) firewalls you implemented.

Phase 5: High-Frequency Surge Capacity (Due: Sunday of Week9)

Business Scenario Aura Digital Bank is executing a major promotional campaign, and Elena expects a massive surge in mobile user registrations. The system must dynamically adapt to workload fluctuations without degrading application performance, dropping connections, or requiring manual infrastructure intervention.

Technical Requirements

To successfully complete this phase, you must:

- **Distribute Traffic:** Establish a resilient entry point that distributes incoming traffic across multiple physical availability zones.
- **Automate Scaling:** Implement a dynamic, automated scaling mechanism that adjusts compute capacity based on system load metrics, such as CPU saturation or active connection thresholds.
- **Configure Health-Checks:** Design custom application health-checks that ensure unhealthy or crashing application nodes are automatically detected, quarantined, and replaced by fresh instances without customer downtime.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Load-Testing Report:** A comprehensive load-testing execution report showing the system successfully provisioning additional compute nodes in response to simulated high-traffic volume.
- **Scale-In Metric Graph:** A metric graph illustrating the automatic return to baseline compute capacity (scale-in) after traffic levels subside.
- **Failover Validation:** An architectural validation showing that your load-balancer endpoint remains reachable and responsive even when one of the backend compute subnets is intentionally simulated to fail.

Note: Regarding the requirement to "simulate high-traffic volume", no specific load-testing software is provided or required. You are free to use any open-source load generation tool from your local machine (such as Apache Bench, JMeter, or Locust), or you can install a simple CPU stress-testing utility directly on your compute instances to artificially trigger the "dynamic, automated scaling mechanism". For the "custom application health-checks", you do not need to build complex diagnostic logic; simply create a basic file in your PHP application (e.g., health.php) that returns a standard HTTP 200 OK status code for the load balancer to query. Finally, to generate the "failover validation" and simulate a backend subnet failure, you can temporarily modify your Network ACLs to block all traffic to a specific Availability Zone, or forcefully terminate all running compute instances within that zone, and then demonstrate that your "load-balancer endpoint remains reachable and responsive" by routing traffic to the remaining healthy subnets.

Phase 6: Banking-as-Code & Automations (Due: Sunday of Week10)

Business Scenario Marcus plans to spin up a mirror environment of Aura Digital Bank in a secondary region to support international operations. To eliminate human error, guarantee compliance consistency, and enforce version control, you must write code to define the entire platform.

Technical Requirements

To successfully complete this phase, you must:

- **Codify Infrastructure:** Codify the entire physical network topology, compute layers, load balancers, database instances, and security groups into Infrastructure-as-Code (IaC) configuration scripts.
- **Use Modular Structures:** Enforce modular coding structures to isolate the base VPC network configuration from the core application stack.
- **Establish CI/CD:** Establish a continuous deployment (CI/CD) system that integrates with a secure git version-control repository, ensuring that any code commits to the main repository automatically trigger safely-planned cloud infrastructure modifications.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Repository Link:** The complete git repository link containing structured, commented IaC modules (e.g., Terraform or AWS CloudFormation).

- **Pipeline Execution Log:** A pipeline execution log proving that a commit to the repository triggered a clean, automated deployment of a network adjustment without manual console clicks.

Note: You are free to choose your preferred IaC tool, such as Terraform or AWS CloudFormation, to "codify the entire physical network topology, compute layers, load balancers, database instances, and security groups". When enforcing "modular coding structures", do not write one giant, monolithic configuration file; you must break your code into separate files, folders, or logical modules to explicitly "isolate the base VPC network configuration from the core application stack". To "establish a continuous deployment (CI/CD) system", you do not need to host your own automation servers; leveraging managed platforms like GitHub Actions, GitLab CI/CD, or AWS CodePipeline connected to your "secure git version-control repository" is highly recommended. Finally, to generate the required "pipeline execution log" proving an "automated deployment of a network adjustment", you do not need to make a massive, risky architectural change. Simply updating a resource tag on your VPC, adding a non-disruptive dummy rule to a security group, or altering a resource description in your code and committing it is sufficient to prove your pipeline triggers correctly without "manual console clicks".

Phase 7: Serverless Reporting & Audit Pipeline (Due: Sunday of Week12)

Business Scenario To prevent money laundering, Maya requires a daily transaction audit report flag and logging of all customer transfers exceeding $10,000. To protect user privacy and preserve primary system database speeds, these heavy analysis jobs must run off-peak, serverlessly, and notify the compliance officer immediately upon identifying high-value transactions.

Technical Requirements

To successfully complete this phase, you must:

- **Schedule Event-Driven Tasks:** Create an event-driven, serverless execution task that executes daily on a fixed off-peak schedule.
- **Secure Private Execution:** Ensure the execution task runs securely inside your isolated private network space to query the database vault, extract matching high-value transactions, and compile a CSV audit report.
- **Implement Alerting:** Establish an immediate, publish-subscribe alerting system that instantly delivers a notification email and the transaction summary report to the compliance team when a high-value transaction is identified.

Expected Audit Evidence & Deliverables

Your engineering handbook must include the following proof of execution:

- **Architecture Diagram:** A visual diagram of the serverless event-driven architecture.
- **Function Code:** The exact code of your serverless function (excluding active secrets).
- **Sample Notification:** A sample email notification received by your testing compliance inbox, complete with the attached simulated audit report.

Note: Regarding the requirement to "extract matching high-value transactions" exceeding $10,000, keep in mind that you do not have a live banking database. You will need to manually insert a few rows of dummy transaction data (making sure at least a couple explicitly exceed the $10,000 threshold) into your isolated database vault from Phase 3 so your serverless function has actual data to query. When writing the code to "compile a CSV audit report", you can generate this file directly in the temporary execution memory (e.g., the /tmp directory in AWS Lambda) or save it to an internal storage bucket before attaching it. For the "publish-subscribe alerting system", you do not need an enterprise email server; simply configure a managed notification service (such as Amazon SNS or SES) and subscribe your own personal email address to act as the "testing compliance inbox". Finally, although the task must execute "daily on a fixed off-peak schedule", you should temporarily configure your event trigger to run immediately (or invoke the function manually) so you can successfully generate the required "sample email notification" for your audit evidence without waiting overnight for the scheduled run.

Phase 8: Audit Defense & Engineering Handover (Due: Sunday of Week 13)

Business Scenario The launch date is here. To receive a banking charter, you must defend your architecture before an external security and compliance audit board.

Technical Requirements

To successfully complete this phase, you must:

- **Compile the Handbook:** Compile a highly professional, cohesive Cloud Architecture Engineering Handbook.
- **Write Executive Summaries:** Deliver a structured executive summary targeting non-technical leadership (Elena) that justifies infrastructure cost-efficiency, alongside a deep-dive security breakdown targeting Maya (CISO).
- **Demonstrate Resilience:** Demonstrate live operational resilience by executing a recorded walkthrough that proves dynamic failovers, secure administrative access, and automated code deployments.

Expected Audit Evidence & Deliverables

For this final stage, your submitted materials must include:

- **Handover Handbook:** A complete, professional PDF or Word handover handbook.
- **Operational Walk-through Video:** An operational walk-through video (maximum 10 minutes) presenting architectural justifications and demonstrating active system validation.

Note: Regarding the requirement to compile a "highly professional, cohesive Cloud Architecture Engineering Handbook", you do not need to generate entirely new technical content; this handbook is simply a polished, unified compilation of all the expected audit evidence, diagrams, and verification logs you have already gathered during Phases 1 through 7. For the "structured executive summary targeting non-technical leadership", keep your justification for "infrastructure cost-efficiency" concise (typically one to two pages) by focusing on high-level concepts, such as how your automated scaling reduces compute costs during low-traffic periods and how serverless reporting eliminates paying for idle servers. When producing the "operational walk-through video", you are not required to show your face on

camera, write a script, or use advanced editing software. You may use any standard, free screen-recording tool (such as OBS, Zoom, or Loom) to capture your voice and screen while you actively demonstrate the required "dynamic failovers, secure administrative access, and automated code deployments". Finally, you must adhere strictly to the "maximum 10 minutes" time limit; it is highly recommended to practice your live system validations beforehand so you can efficiently present your "architectural justifications" without running out of time.

Project Constraints

Throughout all phases of this project, your designs and implementations must strictly adhere to the following cross-cutting constraints:

- **Regulatory Compliance & Zero-Trust Security:** Because Aura Digital Bank is a financial institution, obtaining regulatory clearance (aligning with SOC2 and PCI-DSS standards) is mandatory. You must enforce a zero-trust network topology, physically isolate the database ledger from the public internet, systematically rotate credentials, and ensure all data is encrypted.
- **Automation & Repeatability:** Manual provisioning ("click-ops") and local data silos are strictly prohibited. All infrastructure must be programmatically deployed using Infrastructure-as-Code (IaC) and integrated with automated CI/CD pipelines to eliminate human error and ensure environment consistency.
- **Cost-Efficiency:** To meet the CEO's mandate of minimizing startup infrastructure run costs, you must design resource-efficient solutions. This includes leveraging automated storage tiering to move stale files to archival storage and utilizing serverless, event-driven compute for heavy, off-peak analysis jobs.
- **High Availability & Disaster Recovery:** The platform must survive a complete physical data center outage (Availability Zone failure) without administrative intervention or data loss. Your architecture must distribute traffic across multiple zones, automatically scale compute capacity during massive promotional surges, and maintain a strict disaster recovery strategy.

Note: Regarding the due dates mentioned throughout the project phases, please be aware that the week numbering is based strictly on the official TRU calendar. The first week of the semester is officially considered Week 1, even if the first day of classes falls in the middle of a calendar week. Please ensure you calculate your phase submission deadlines accordingly.

Teamwork

This project will be completed in groups of 2-3 students. You must inform the instructor of your team members via email by the **end of the first week**. For students who have not formed a team, the instructor will randomly create groups and post them on Moodle on the first day of the second week.

The most important part of a successful team is finding a time when everyone meets, as a group, each week, for at least two hours. Each student should expect to spend about four hours outside of class per week on the project. Compare your schedules to find a time block of at least two hours when you all can meet. Commit to meeting at this time each week and working together. If something comes up and someone can't make it, be sure to schedule another time for that week.

You will deliver your team charter during the first phase of the project. A team charter is a document of your own design. It should be "artfully" designed, expressing some interests and passions of your team. Things to discuss: What are your goals for the class? Who are your team members? What is your team mascot? How will the team celebrate triumphs? How will the team make important decisions? How will the team resolve conflicts and discuss problems? What does "leadership" mean to your team? Who is the person that hits "submit" on your reports and milestones? What are the skills of the team members? When will the team meet as a group each week? (Please be precise.) What will be the procedure for missing or being late to this meeting? How much advance notice must be given and using what method? A team charter template has been uploaded alongside the project files.

Part 3: Deliverables, Handover & Grading Rubric

Each team must compile their architecture specifications, verification logs, configurations, and diagrams into a single, professional engineering design book. The project will be graded on a 100-point scale according to the following strict structural rubric:

Assessment Component	**Evaluation & Quality Standard Criteria**	**Weight**
Phase Completion (Phases 1-8)	Successful execution and completion of the 8 core engineering phases (5% per phase).	40%
Architectural Soundness & Design	How effectively the proposed architecture satisfies constraints (multi-tier isolation, serverless boundaries, zero-trust network topology, secure machine identity). No default settings used.	10%
Compliance & Perimeter Defense	Integration of S3 compliance locks, MFA delete, encrypted traffic channels, secrets management (Ssm/Secrets Manager), and secure administrative access without public internet ports.	10%
Operational Resilience & IaC	Verification of high availability under simulated load testing, automatic scale-out/scale-in patterns, automated infrastructure reproducibility using Terraform/CloudFormation modules, and CI/CD triggers.	10%

Technical Runbook & Handover Video	Completeness of the engineering handover documentation, code formatting, clear administrative guidelines, and a cohesive, professional 10-minute system walk-through and verification video. **The runbook must be written in the official course project template.**	15%
Audit Evidence & Verification	Quality and validity of expected audit evidence, terminal command outputs, screenshot logs of active deployments, sample alerts, and physical/logical architecture diagrams.	10%
Presentation	Clarity, professionalism, and the team's ability to effectively communicate complex technical concepts and defend their architectural design decisions.	5%

Academic Integrity & Artificial Intelligence (AI) Usage Policy

In this course, we recognize that Generative AI tools (such as ChatGPT, Gemini, GitHub Copilot, etc.) are powerful, standard resources in the modern tech industry. However, the primary goal of this project is for *you* to develop critical thinking and practical skills in cloud architecture and engineering. Therefore, the use of AI is governed by the following guidelines:

1. Fully Permitted Uses (Out of Scope Components) You are fully permitted to use AI tools to generate, debug, or optimize project assets that are outside the core scope of cloud computing. This includes:

- **Web Design:** Generating HTML, CSS, or frontend assets for the Phase 1 marketing portal.
- **Application Code:** Writing or modifying the PHP application logic for the Phase 2 ledger prototype.
- **Databases:** Creating SQL schemas, dummy data, or queries for the Phase 3 database migration.

2. Permitted as a Learning Assistant (In-Scope Components) For core cloud computing tasks (such as network topology, IAM policies, Infrastructure-as-Code, and serverless architectures), you may use AI as an interactive tutor. You can use it to:

- Explain complex cloud concepts (e.g., "Explain how BGP routing works in a hybrid cloud").
- Help troubleshoot specific terminal error messages or pipeline failures.
- Provide examples of syntax for Terraform, CloudFormation, or AWS CLI commands.

3. Prohibited Uses (Academic Misconduct) You must not use AI to bypass the engineering and critical thinking required for the core deliverables. Specifically, you may **not**:

- Prompt an AI to generate your entire Infrastructure-as-Code (IaC) configuration or network topology from scratch.
- Use AI to write your Cloud Architecture Engineering Handbook, executive summaries, or security justifications.
- Submit AI-generated architecture diagrams as your own original work.

If you use AI to assist with any part of your core infrastructure code, **you assume full responsibility for its output**. AI frequently generates outdated, inefficient, or insecure cloud configurations (e.g., utilizing default VPCs or overly permissive IAM roles), which will result in heavy point deductions based on the grading rubric. During the Phase review sessions, you must be able to confidently explain, justify, and defend every line of code, security policy, and architectural decision in your project.

Declaration Requirement Your final engineering design book must include a brief **AI Usage Declaration** section. You must transparently list which AI tools were used during the project and explicitly describe how they were utilized (e.g., "Used ChatGPT to generate the PHP ledger application and to explain AWS NACL rule numbering").



THOMPSON RIVERS UNIVERSITY

SENG 4620 - Practical Cloud Computing

Project Title

Team member name (TRU ID)

Team member name (TRU ID)

Team member name (TRU ID)

Date submitted

Table of Contents

1 Executive Summary 6

2 Introduction 7

3 Design Problem 7

3.1 Problem Definition 7

3.2 Design Requirements 7

3.2.1 Functions 7

3.2.2 Objectives 7

3.2.3 Constraints 7

4 Solution Development 8

4.1 Solution 1 8

4.2 Solution 2 8

4.3 Final Solution 8

4.4 Components & Features 8

5 Prototype Development and Testing 9

5.1 Initial Testing 9

5.2 Report on Final Testing 9

5.3 Environmental, Societal, Safety, and Economic Considerations 9

5.3.1 Safety Analysis - to identify possible hazards in the design and evaluate the risk that could lead to an accident 9

5.3.2 Economic Considerations 9

5.3.3 Life Cycle and Environmental Impact Analysis 10

5.4 Limitations 10

6 Team Work 11

6.1 Meeting 1 11

6.2 Meeting 2 11

6.3 Meeting 3 11

6.4 Meeting 4 11

7 Project Management 13

8 Conclusions 14

10 References 15

11 Appendix.....16

- The table of contents should be automatically generated by selecting "References/ Table of Contents". Remember that the table of contents should not have an entry of the "Table of Contents" itself.
- Proof read the text for typing and grammar mistakes.
- Follow the IEEE Bibliography style for the references by selecting "References/ Citations & Bibliography/ Style".
- See the guide in the link for more details:
- https://ieeeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf

List of Figures

- Similar to table of contents, list all figures in the report with page numbers.

List of Tables

- Similar to table of contents, list all tables in the report with page numbers.

1 Executive Summary

The summary is considered by many to be the most important part of a report. It is a difficult part to write and should always be written last. It is often the only part that managers in the work place have time to read, so it is important to state your case succinctly (normally less than one page).

Purpose: Provides a brief summary of the report, an information abstract. No reference is made to any part of the report; a summary is complete in itself.

Content:

- states the more important information in the report
- the purpose, method, reason for the report
- the main findings, conclusions, decisions, and recommendations
- all major generalizations or assumptions of the report
- There shouldn't be any references in the executive summary. The research and background go in the introduction of this report.

Note about the report: The length of your report and each section should reflect the complexity of the topic and the thoroughness of the research and design. The report should be self-contained (any terminology particular to the topic should be defined the first time you use it), consistent, and to the point. It should be understandable to someone who has background in the area of the report but is unfamiliar with the particular topic of the report. Use standard formal level of English (no slang or colloquialisms)

2 Introduction

- Introduces the subject of the report to the reader. Remember that the reader may be from a different branch of the discipline, and will require some orientation to the subject of your report.
    - Note that for this report, there is no background section, so the background is combined with the introduction. Start the introduction section by showing some research and background on the topic of your subject. Make sure that you include references for everything in the report that is not your own thoughts, and that everything is referenced in IEEE style. For IEEE, put a number (i.e. [1]) next to the referenced phrase or figure title, and then be sure to state the reference in your reference section. Microsoft word can do this for you- use "references" tab, then "insert citation".
- Should contain the subject and purpose of the report: states briefly why the report is being written and what the re- port is intended to achieve.
- Scope: describes how broad or how limited the treatment of the subject will be
- Plan of development: outlines which areas will be covered
- Thesis of the report: the general conclusion and/or the general recommendation

3 Design Problem

Open each section of the report with an introductory paragraph (at least a few sentences) on what the section is about.

This section has the following two subsections:

3.1 Problem Definition

Write a problem statement of the project.

Describes the "gap" (i.e., need) that you are seeking to fulfill with the solution to the problem.

3.2 Design Requirements

This section has the following two subsections:

3.2.1 Objectives

- Provide objectives of the design project. Remember that the objectives are specified as adjectives.
- Include an Objective tree. Remember that objectives are attributes that the final design should possess- such as safe, economical, portable, etc. Each primary object should have more detailed objectives below it (sub-objectives).

3.2.2 Constraints

- Provide constraints here. Remember that the constraints are binary (either satisfied or not). Constraints are used to verify if a solution can be considered or discarded.

4 Solution Development

- Provide multiple, valid, different solutions to the problem. Each design should represent a different approach to the problem, a different set of trade-offs between the objectives.
- Enable the reader to have a clear picture of each alternative as well as its strengths and weaknesses in order to compare them.
- All alternatives must be feasible.
    - There should not be one clearly superior design and several clearly inferior designs,
    - nor should the alternative designs be merely components of a single design that emerges as the proposed design in the end.

4.1 Solution 1

Gives, in one or two paragraphs, a short description of the alternative design.

Reviews, in bullet lists, advantages and disadvantages with respect to project requirements, such as objectives and constraints.

Provides some preliminary sketches, drawings, and/or diagrams (e.g., flowchart or use case diagram) of the design, with measurements where appropriate; extra figures may be put in the appendices. Refer your reader to these appendices clearly in the body of the report, identifying each appendix by alphabetical designation and title (e.g., Appendix A: Design Option 1—Details of the Power Supply).

Uses parallel structure to make it easier for the reader to understand the comparison. Parallel structure means structuring the description of each design alternative the same way, with information given in the same order using the same formatting.

*Below each figure, figures should be numbered and labelled as Figure 1, 2, etc. followed by the figure title.

*Tables should be numbered and labeled above each table as Table 1, 2 etc., followed by the table title.

**Be sure that before a figure or table is placed in the report, there is a discussion about the figure or table, and what it shows. (Don’t just plop figures or tables into the report without out some introduction/commentary about them first).

4.2 Solution 2

As above for Solution 1, but for a second alternative design.

4.3 Final Solution

This is the final solution. Explain why it is better than other solutions. You may use a table for comparison purposes.

4.4 Components & Features

Describe each of the components and features for each of the parts of the final design in detail.

Refer to SolidWorks diagrams as each component is described.

Use the correct engineering terminology and describe the physical process and energy transformations that are powering the motion in each step. For example, chemical energy in a battery is transformed to electrical energy and then to mechanical energy to turn something. Types of energy include, but are not limited to: gravitational, kinetic (the energy an object has due to it's motion), elastic, thermal, sound, chemical, and electrical.

5 Prototype Development and Testing

.

5.1 Initial Testing

What have you used to test your design to ensure that it meets the objectives?

What improvements were made during the testing and evaluation of the prototype?

Photographs (figures) of the testing process may be helpful to explain things.

5.2 Report on Final Testing

The purpose of this section is to provide as detailed evidence as possible that the proposed design successfully meets the project requirements. The section is written as a table and indicates:

- How have you tested your proposed design to ensure that it meets the objectives.
- Clear measures of success
- The section can be written as a table preceded by an introductory paragraph.

Photographs of the final testing process can provide useful visualization aids. Be sure to also submit a video of the prototype during the development process.

5.3 Environmental, Societal, Safety, and Economic Considerations

Explain how your engineering design considered environmental, societal and economic considerations. What type of economic decisions did you make? How did you make sure that the design is safe to use, etc.

5.3.1 Safety Analysis - to identify possible hazards in the design and evaluate the risk that could lead to an accident.

- An introductory paragraph
- Hazards of the design, considering how they may be reduced or eliminated or how safer alternatives might be used.
- Accidents and how humans and property may be guarded.
- Warnings and training for the operators

5.3.2 Economic Considerations

- Detailed costs of each of the purchased components and how well the project remained under budget. A table to break this down may be helpful.
- Discuss where the other components came from. This leads into the next section- life cycle analysis.

5.3.3 Life Cycle and Environmental Impact Analysis

- Provide a life‑cycle analysis; a 3 R's approach - reduce, reuse and recycle.
- Some background research on the life cycle of the items used for your project would be helpful to show that this was considered in depth. (Be to reference your sources).
- Discuss how the design tries to lessen the negative impacts and increase the positive ones.
- Recommendations for construction/design/implementation/operation/de-commissioning to reduce environmental impact.

5.4 Limitations

Every product has some limitations, and so is the case with your design product. Highlight some of the limitations of your prototype here.

6 Team Work

Provide our group roles and responsibilities. You may past in components of Deliverable #1. Provide the details of each of your meetings, as per the below example.

6.1 Meeting 1

Time: September 22, 2020, 10:00 am to 11:00 am

Present: A list of people attend the meeting

Agenda: A list of topics to be discussed in the meeting

Meeting minutes:

A list of decisions made in the meeting. Readers should be able to understand the whole design process by reading the meeting minutes. It should cover:

- Detailed task assignments – using the table below.

Team Member	Previous Task	Completion State	Next Task
Team member 1	N/A	N/A	Task 1
Team member 2	N/A	N/A	Task 2
Team member 3	N/A	N/A	Task 3
Team member 4	N/A	N/A	Task 4

- A list of discussion items – show major design-related discussions separately, under each item, explain program background, the solution chosen and reasons for the choice.

Record of team rule penalty: A list of actions by team members which resulted in warnings or penalties. Specify who receives a warning or penalty, the reason and when they were notified.

6.2 Meeting 2

Provide a similar description here.

6.3 Meeting 3

Provide a similar description here.

6.4 Meeting 4

Provide a similar description here.

Note in the Last meeting to add a summary of Team rule penalty records as follow:

Scores	Team member 1	Team member 2	Team member 3	Team member 4

Name				
Student ID				
Total mark				
Notes and Comments				

7 Project Management

Provide a Gantt chart showing the progress of your work here.

- Elements to be included: • Dates. • Tasks. • Bars. • Milestones. • Arrows (predecessors). • Task ID. • Resources. • Durations.
- Provide the slack time of each task
- Each primary task should have sub-tasks and each should be assigned to/accomplished by an individual.

Discuss the how well project management went.

- What were the keys to success?
- What do you think your team could do better next time?

8 Conclusions

- A summary of what you achieved. Mention all the design functions and objectives that you achieved while satisfying the constrains.
- Summarizes the key points of what you did (steps along the way and final outcome), and brings the document to a close.
- What did you learn from this experience?
- Not a word-for-word repetition of earlier material

9

10 References

- Use the IEEE reference style.
- Do not put in any references that are not cited in the text.

11 Appendix

- If you want to provide any additional information, use this appendix.
- Include data that is not necessary to an immediate understanding of the discussion.
- Appendices can contain drawings, calculations, technical specifications, or other detailed explanations of some aspects of your report.

THOMPSON RIVERS UNIVERSITY

SENG 4620 - Practical Cloud Computing

Group Work Challenges and Solutions

Group work challenges	Description	Solution
Scheduling problems	This can create roadblocks to getting started/continuing with projects. It is frustrating for students who feel that others aren't compromising and don't take their situation into consideration.	● Try and be understanding of others' schedules and responsibilities which may be different from your own. ● Consider using virtual meeting spaces such as messenger chats, Skype, and email to communicate. ● Take turns picking the venue and time of the meeting.
Group conflict	Group conflict is natural and often necessary for effective projects. Sometimes though, it escalates and makes it even more difficult to focus.	● Don't let personal feelings impact your work in the group. Focus on the task. ● Try and find common ground between two ideas to reach reconciliation. ● Address conflicts directly and respectfully.
Uneven contribution	● Some group members don't contribute to the group project or aren't perceived to be contributing to the group. ● This creates tension in the group and is unfair to the group members	● Set up clear guidelines and work expectations at the beginning of the group project. ● Assign roles and responsi-bilities so that each person will be making an equal contribution. ● Speak directly, but respectfully to the person who is not completing their work.

Group work challenges	Description	Solution
Different expectations	• Some members strive for perfection, while others simply want to pass. Some begin projects in advance, while others procrastinate. • This can create tension because the group is not working towards the same goal.	• Early communication is key to make sure everyone is focused on common goals. • Keep goals realistic and understand that your actions affect others in the group. • Make a timeline so that your group can stay on an agreed plan for getting the project done.
Getting stuck	At some point groups may get 'stuck' and hit a mental road block. This is discouraging and can lead to procrasti-nation and avoidance.	Reread project expectations and goals. Have a brainstorming session where ideas are discussed. Create a mind map to link common ideas and trains of thought. Seek help from your professor if you remain stuck.
Groupthink	'Groupthink' occurs when members of a group agree with other group members in order to avoid conflict. This stifles creativity and constructive evaluation of alternative ideas.	Thinking critically about ideas presented, offering and assessing alternatives, and embracing diverse opinions from group members. Work through projects analytically using the groups' combined knowledge and experience.

References:

Working Effectively in Groups (2021). Learning Services, University of Waterloo. Available at: https://uwaterloo.ca/student-success/sites/ca.student-success/files/uploads/files/TipSheet_GroupWork_0.pdf (Accessed: 12 April 2021).

THOMPSON RIVERS UNIVERSITY

SENG 4620 - Practical Cloud Computing

Group Contract Guidelines

This document contains guidelines and clauses that your group may wish to use to form a group contract for your SENG 4620 Term Project. Do not copy this document exactly. It is a guideline. Your group should make their own and tailor it to what you agree will work best for your team.

The success of the group will depend on the cooperation and professionalism of its members. Employers will expect you to know how to work effectively in groups: how to determine what needs to be done; how to find information; how to assess information; how to share the workload; and how to resolve interpersonal conflicts that might arise. Effective collaboration includes, but is not limited to:

- Participating fully (in spirit and actuality)
- Participating professionally (i.e., civil discourse; abiding by the rules of academic honesty)
  Meeting responsibilities (i.e., completing assigned tasks on time and to the best of your ability)
- Taking the consequences of not abiding by the group's rules.
  Giving group members appropriate credit where due
- Not giving credit where it isn't due

After reading through this document, select some or all of the rules you would like to implement as a group, and add any other that you feel would be beneficial. If you disagree with these rules, then as a group, amend them. It is important to set mutually agreed upon ground rules for contribution. For example, if you miss a meeting you could be expected to read and respond to notes taken at the meeting within 24 hours. Or, if you consistently miss meetings/fail to communicate/produce work, you will no longer be a part of the project group and will be expected to complete the group project on your own. (The instructor must be informed of this). That being said, life happens. Put yourself in the shoes of your group members. Try and understand where they are coming from and be inclusive as much as possible.

1. Each group member agrees to show up to class and to outside group meetings on time.

   Initials:

2. In the event that a group member is less than five minutes late, s/he may *quietly* join the group without disrupting it to ask what s/he missed. It is optional for the group members to fill in the late-comer.

   Initials:

3. Group members who are avoidably late must: [write out a consequence for being late.]

Initials:

4. A group member who is absent more than_____times will be dismissed from the group.

   Initials:

5. If a member submits plagiarized material and/or cheats, the group agrees to bring this to the instructor's attention immediately.

   Initials:

6. Each member agrees to familiarize him- or her- self with and abide by TRU's rules for Academic Honesty (available on the course Moodle site).

   Initials:

7. Members agree to treat one another with respect. Respect includes no name-calling. If you don't like an idea, address the idea, not the person (for example, "I don't think that idea will work because…" not "That's stupid"). In the event that a group member treats someone inappropriately, s/he will [write a consequence]:

   Initials:

8. No "cross talking" is allowed. This means not interrupting when someone else is talking.

   Initials:

9. In the event that a group member or members are dominating the group, it's the time keeper's job (or whoever you have assigned to this role) to politely interrupt them (this is when you *can* interrupt) and ask that someone else speak.

   Initials:

10. Other rules that the group would like to add:

Each member print name & then sign: Date:_____

Print Name: Signature:

Print Name: Signature:

Print Name: Signature:

References:

Sample Group Contract (2021). Teaching, Learning & Technology Center/Division of Undergraduate Education, UC Irvine. https://cns.utexas.edu/images/CNS/TIDES/teaching-portal/Examplegroupcontract.pdf (Accessed: 12 April 2021).

Working Effectively in Groups (2021). Learning Services, University of Waterloo. Available at: https://uwaterloo.ca/student-success/sites/ca.student-success/files/uploads/files/TipSheet_GroupWork_0.pdf (Accessed: 12 April 2021).